تمرین اول – گروه ۲۲

آدرس مخزن: https://github.com/phatemek/Software-Testing

شناسه آخرین کامیت: ada7e85727aaf32a47902916184c8bda49acff07

سوالات تئوری

۱. متدهای Assume در JUnit زمانی استفاده می‌شوند که بخواهیم یک شرط خاص را قبل از اجرای تست بررسی کنیم. اگر شرط برقرار باشد، تست اجرا می‌شود و اگر شرط برقرار نباشد، تست نادیده گرفته می‌شود. (به جای fail شدن skip می‌شود.) متد AssumeTrue به‌طور خاص زمانی استفاده می‌شود که بخواهیم بررسی کنیم آیا یک شرط بولین true است یا خیر. اگر نتیجه شرط true باشد، تست ادامه می‌یابد و در غیر این صورت تست رد می‌شود. این متد برای زمانی مفید است که تست وابسته به شرایط خاصی باشد و اگر آن شرایط برقرار نباشد، تست بی‌معنی باشد.

۲. شرایطی که موجب پیش آمدن اتفاقات غیرقابل قبول و یا پاسخ‌های نادرست در اجرای یک برنامه به صورت Multi-threaded می‌شود (deadlock, race condition, ...) عموما شرایط پیچیده و دارای randomness زیاد می‌باشند. برای مثال کوچک‌ترین اختلاف‌ها بین زمان اجرای برنامه توسط thread ها(که امری غیر قابل کنترل در unit test ها می‌باشد) می‌تواند باعث به وجود آمدن یا نیامد یک deadlock یا race condition شود و بنابراین صرفاً با اجرا کردن برنامه به صورت Multi-threaded در unit test ها نمی‌توانیم تضمین کنیم که آن شرایط پیچیده را پدید می‌آوریم و همواره ممکن است در یک بار اجرای تست مشکلی پیش نیاید و در اجرای بعدی دچار مشکل شود.

۳. در صورتی که به جای استفاده از assert، خروجی را در کنسول چاپ کنیم، به ازای هر unit test مجبور می‌شویم درست بودن پاسخ آن را خودمان از طریق کنسول چک کنیم. این کار علاوه بر گرفتن زمان زیاد، ممکن است دچار خطا باشد به این دلیل که ممکن است بعضا خروجی تولید شده بسیار شبیه خروجی مورد انتظار باشد و ما متوجه خطای آن نشویم. همچنین در بعضی تست‌ها خروجی‌هایی تولید می‌شوند که به سادگی قابل چاپ در کنسول نیستند. برای مثال هنگامی که خروجی یک object است چاپ کردن و چک کردن آن کار ساده‌ای نیست و با استفاده از assert بسیار ساده‌تر انجام می‌شود.

۴.

الف) در این تست باید از Assertions.assertThrows استفاده کرد. در صورت استفاده از expects Exception تنها می‌توان مطمئن بود که کل تست یک استثنا پرتاب می‌کند اما مشخص نیست که دقیقا کدام بخش از کد آن را پرتاب کرده است. این باعث می‌شود اگر استثنا در جایی غیرمنتظره از کد رخ دهد، تست همچنان موفق باشد، در حالی که باید شکست می‌خورد. همچنین در استفاده از expects Exception نوع دقیق استثنای مورد انتظار مشخص نیست. کد جایگزین به این شکل است:

```
@Test
void testB() {
        int badInput = 0;
        Assertions.assertThrows(IllegalArgumentException.class, () -> {
                new AnotherClass().process(badInput);
        });
}
```

ب) در این کد از یک شی Calculator ثابت برای چند تست استفاده شده است که ممکن است باعث مشکلاتی در تست‌های متوالی شود (به علت وضعیت تغییر کرده شی). برای حل این مشکل باید در هر تست یک شی جدید ایجاد شود یا وضعیت شی reset شود یا از متد beforeach استفاده شود:

```
class CalculatorTest {
        private Calculator fixture;
        @BeforeEach
        void setUp() {
            fixture = Calculator.getInstance();
            fixture.setInitialValue(0);
        }
        @Test
         void testAccumulate() {
            int result = fixture.accumulate(50);
            assertEquals(50, result);
        }

        @Test
        void testSubsequentAccumulate() {
            int result = fixture.accumulate(100);
            assertEquals(150, result);
        }
}
```